



Università degli Studi dell'Insubria

Dipartimento di Scienze Teoriche e Applicate

Corso di laurea triennale in Informatica

Tesi di Laurea

**Progettazione e sviluppo di un market
su rete basata su microservizi tramite
la piattaforma Kubernetes**

Relatore

Ch.ma Prof.ssa Sabrina Sicari

Co-relatore

Prof.ssa Alessandra Rizzardi

Tesi di Laurea di

Federico Garegnani

Matricola 746789

Anno accademico

2022–2023

Indice

1	Introduzione	1
2	Stato dell'arte	3
2.1	Architettura monolitica	3
2.2	Deployment tradizionale	4
2.3	Deployment su macchine virtuali	5
2.4	Deployment su container	6
3	Architettura del sistema	7
3.1	Struttura	7
3.2	Tecnologie utilizzate	8
3.2.1	Python	8
3.2.2	PostgreSQL	8
3.2.3	Docker	8
3.2.4	Kubernetes	9
3.2.5	API REST	10
3.2.6	Token JWT	11
4	Implementazione	12
4.1	Autenticazione e autorizzazione	12
4.2	Database	13
4.3	Esecuzione delle richieste	14
4.4	Evoluzione dello stato di un ordine	15
4.5	Pagamento di un ordine	17
4.6	Implementazione dei microservizi	17
4.7	Altre funzionalità	20
5	Risultati ottenuti	22
6	Conclusioni e sviluppi futuri	26
	Bibliografia	29

1

Introduzione

Lo sviluppo di applicazioni distribuite rappresenta una disciplina fondamentale nell'evoluzione del mondo dell'informatica. Questa pratica si concentra sulla progettazione ed implementazione di sistemi software che operano su reti di computer interconnessi, consentendo la condivisione efficiente di risorse e dati. All'interno del panorama tecnologico odierno lo sviluppo di applicazioni distribuite risulta cruciale per soddisfare le esigenze di sistemi complessi.

In ambito di sviluppo delle applicazioni suddette si riconoscono due grandi modalità di approccio al problema: un approccio più tradizionale che vede il sistema da sviluppare come un'unica unità, ed un metodo più moderno che mira a frammentare il problema principale in unità di minori dimensioni cooperanti tra loro. Il primo approccio descritto segue un'architettura monolitica e prevede che l'intera applicazione sia sviluppata e distribuita come un unico componente; contrapposta a questa, negli ultimi anni è emersa l'architettura a microservizi, un approccio architetturale che mira a scomporre la logica applicativa in componenti piccole ed autonome, consentendo lo sviluppo ed il rilascio indipendente dei singoli servizi[1].

L'approccio a microservizi porta con sé una serie di vantaggi:

- *Scalabilità.* Possibilità di scalare singoli servizi in base alle esigenze, senza influenzare l'intera applicazione.
- *Indipendenza.* I diversi team possono lavorare in modo autonomo su diversi microservizi.

- *Adattabilità.* È possibile sfruttare tecnologie differenti per ciascun microservizio, adattandosi meglio a requisiti o compiti specifici.
- *Manutenibilità.* Un problema ad un microservizio non si propaga all'intera applicazione ma rimane isolato all'interno di esso, facilitandone l'individuazione.

Obiettivo di questo progetto è di sviluppare un semplice sistema gestionale di un supermercato secondo i due approcci sopra descritti e confrontarne infine le prestazioni.

L'applicazione prevede l'interazione con i clienti che acquistano prodotti dal supermercato e, per ogni loro acquisto, implementa la gestione dell'ordine: dalla rimozione dei prodotti dal magazzino fino all'avvenuto pagamento. È possibile vedere il servizio complessivamente offerto come l'e-commerce di un supermercato.

Sono definite due categorie di utenti per il servizio:

- *Utente standard.* È interessato all'acquisto dei prodotti e genera nuovi ordini.
- *Utente fornitore.* Si occupa di rifornire periodicamente il magazzino.

Gli ordini seguono questo percorso:

- a. *Creato.* L'ordine è stato ricevuto dal sistema;
- b. *Confermato.* È stato confermato che nel magazzino ogni prodotto è presente in quantità sufficiente per soddisfare l'ordine;
- c. *Pagato.* Il pagamento dell'ordine è andato a buon fine;
- d. *Spedito.* L'ordine è stato spedito ed è possibile verificare lo stato della spedizione.

Nei capitoli seguenti verranno descritti dapprima lo stile architetturale monolitico e a microservizi, li si confronterà e si elencheranno pro e contro, per poi concentrarsi nello specifico sull'applicazione sviluppata.

In dettaglio:

- *Capitolo 2:* stato dell'arte nello sviluppo di applicazioni distribuite, relativi vantaggi e svantaggi di ogni approccio.
- *Capitolo 3:* architettura del sistema e tecnologie impiegate.
- *Capitolo 4:* spiegazione delle due soluzioni proposte.
- *Capitolo 5:* confronto di performance tra le due soluzioni.
- *Capitolo 6:* conclusioni e possibili sviluppi futuri.

2

Stato dell'arte

In ambito di sviluppo e deployment di applicazioni distribuite si distinguono diversi metodi. Nel corso di questo capitolo verranno analizzati pro e contro di ogni approccio.

2.1 Architettura monolitica

L'architettura monolitica è il modello tradizionale di sviluppo di un'applicazione in cui l'intera logica applicativa è implementata in un unico componente. Un simile approccio è ideale per applicazioni di dimensioni ridotte e che non devono soddisfare un elevato numero di richieste, presentando infatti una minore complessità di sviluppo ed offrendo performance migliori[10]. All'aumentare della dimensioni dell'applicazione o del traffico da soddisfare il modello monolitico inizia però a mostrare i propri limiti:

- *Il sistema non è scalabile:* Se una singola parte dell'applicazione è soggetta a un carico di lavoro elevato, questa non può essere ridimensionata individualmente, ma si è obbligati a scalare l'intera applicazione.
- *Poca flessibilità:* Tutti i moduli devono obbligatoriamente utilizzare la stessa tecnologia, una modifica in questo senso comporterebbe la riscrittura di buona parte dell'applicazione. Alcuni vincoli potrebbero essere: la scelta di un linguaggio di programmazione unico, di un framework specifico, di un unico sistema per la gestione del codice sorgente o di un'unica piattaforma di hosting.

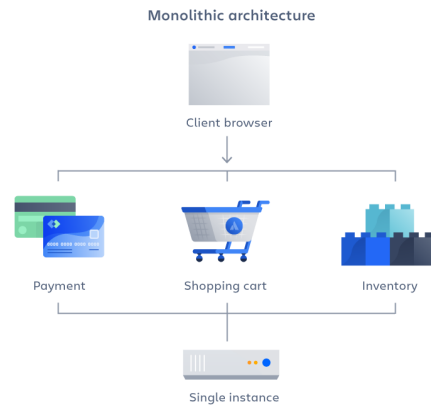


Figura 2.1: Schema di funzionamento di un'applicazione monolitica[10]

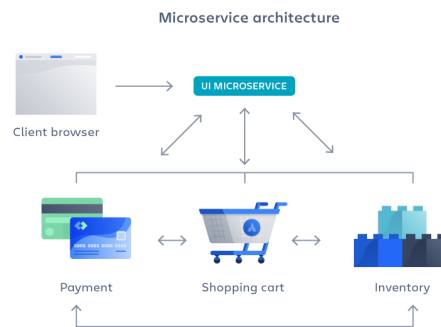


Figura 2.2: Schema di funzionamento di un'applicazione a microservizi[10]

- *Poca resilienza*: Un errore all'interno di un modulo può portare al blocco dell'intera applicazione.
- *Mantenimento*: Anche una singola modifica all'interno di un modulo costringe ad eseguire nuovamente il deployment dell'intera applicazione.

2.2 Deployment tradizionale

La metodologia tradizionale di deployment prevede l'esecuzione delle applicazioni direttamente sul server fisico. Sebbene semplice ed immediato questo modo di procedere non è ottimale, non permette infatti di scalare l'applicazione ponendo dei limiti sull'utilizzo

di risorse (CPU, RAM, disco, ...) e si corre il rischio che una seconda applicazione non riesca a garantire le prestazioni desiderate. Si potrebbe cercare un'alternativa a questo problema eseguendo ogni applicazione su un server fisico distinto, ma anche una tale soluzione non è adatta a un contesto reale risultando molto costosa e portando a un sottoutilizzo delle risorse complessive[1].

2.3 Deployment su macchine virtuali

Le macchine virtuali sono pacchetti software piuttosto pesanti, in grado di eseguire l'emulazione dei dispositivi hardware di basso livello quali CPU, RAM, dischi e schede di rete. Il deployment su macchine virtuali (Figura 2.3), anche chiamato deployment virtualizzato, prevede l'esecuzione di più macchine virtuali sul server fisico, ciascuna delle quali esegue un sistema operativo che a sua volta esegue una o più applicazioni. Un simile approccio è in grado di far fronte ad alcune delle problematiche del deployment tradizionale:

- *Limitazione delle risorse:* In funzione di quanto detto sopra è possibile limitare le risorse dedicate ad una macchina virtuale in termini di tempo di CPU, quantità di RAM, etc.
- *Isolamento:* Le macchine virtuali vengono eseguite in un ambiente isolato e completamente autonomo che le rende immuni da tentativi di interferenza da parte di altre macchine virtuali attive sullo stesso host.

Contrariamente, i principali svantaggi delle macchine virtuali riguardano la ridotta velocità di generazione e distribuzione delle stesse e il maggiore spazio di archiviazione richiesto: trattandosi di *immagini* di un sistema indipendente possono infatti richiedere diverso tempo per essere generate e configurate, l'affiancamento di diverse macchine virtuali porta inoltre alla duplicazione del sistema operativo installato su di esse, con un conseguente utilizzo di spazio di archiviazione.

Esistono un gran numero di piattaforme online che permettono l'acquisto di macchine virtuali in cloud, permettendone di dimensionare ciascuna macchina sulla base delle esigenze del cliente. I servizi più famosi sono senz'altro Amazon Web Services (AWS), Microsoft Azure e Google Cloud Platform (GCP).

Negli ultimi anni sono anche nati diversi sistemi che permettono di automatizzare la generazione, la configurazione e la gestione di macchine virtuali; essi permettono di dichiarare lo stato di esecuzione che si desidera avere e si occupano di mantenere questo stato nel tempo. Rientrano in questa categoria strumenti quali Ansible, Vagrant e Terraform.

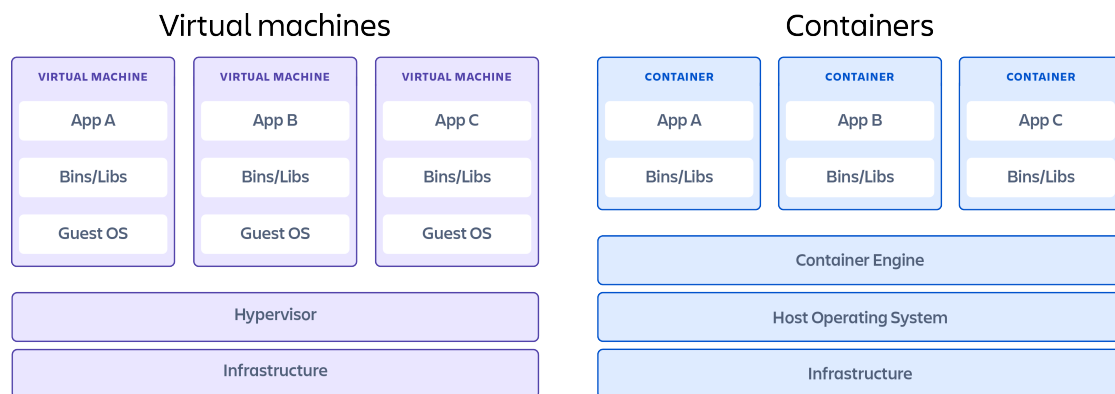


Figura 2.3: Confronto tra un sistema basato su macchine virtuali e container[11]

2.4 Deployment su container

Un *container* è un modello di virtualizzazione più leggero rispetto ad una macchina virtuale che racchiude un software e tutte le dipendenze necessarie ad eseguirlo. Al pari delle macchine virtuali i container rappresentano un ambiente isolato, con risorse limitate ed un proprio file system per l'esecuzione del software ma non virtualizzano un intero sistema operativo (Figura 2.3). L'uso di container offre una grande efficienza in fase di sviluppo e di esecuzione, contengono infatti solo software di alto livello e sono molto veloci da modificare, oltre al fatto che non dovendo virtualizzare un sistema operativo la loro esecuzione risulta molto più rapida e meno esosa di risorse hardware. Per finire molti sistemi runtime per container mettono a disposizione degli sviluppatori repository pubbliche con immagini preconfezionate di software popolari, il tutto al fine di avere uno sviluppo ancora più rapido (Figura 2.2) [1, 11].

Docker è sicuramente il più popolare tra i sistemi sviluppati per offrire la virtualizzazione su container, ed è anche merito di questo progetto *open-source* se lo sviluppo su container si è diffuso così tanto negli ultimi anni. Spesso utilizzato come alternativa a Docker è Podman (Pod Manager), il quale pone maggior accento sull'aspetto della sicurezza[12].

3

Architettura del sistema

Nel corso di questo capitolo verrà dapprima posta l'attenzione sulla struttura dei due sistemi sviluppati, per poi focalizzarsi sulla descrizione delle tecnologie utilizzate.

3.1 Struttura

Come già precedentemente evidenziato il sistema è stato sviluppato seguendo due filosofie differenti, dapprima secondo un approccio monolitico, in seguito seguendo un approccio a microservizi; le due applicazioni sono tra di loro molto simili, condividono buona parte del codice e sfruttano le stesse tecnologie.

L'applicazione sviluppata in modo monolitico espone alcune chiamate API REST, che le permettono di ricevere richieste ed inviare risposte agli applicativi lato client attraverso il protocollo HTTP. La condivisione di informazioni tra client e server avviene attraverso lo scambio di messaggi in formato JSON ed è protetto con algoritmi di crittografia e metodi di autenticazione tramite token, il tutto per garantire la sicurezza e la privacy dei dati scambiati. Per la conservazione dei dati persistenti (quali informazioni sugli utenti, sulla merce in magazzino o sugli ordini) si utilizza infine un database PostgreSQL.

Il sistema a microservizi nasce dall'applicazione monolitica e la scompone in un numero di unità pari ai servizi complessivamente offerti, nel contesto applicativo ciascun modulo si riduce così ad esporre una singola chiamata API, occupandosi di un unico aspetto tra i passi successivi che portano dalla scelta dei prodotti fino alla spedizione

dell'ordine. Il sistema si compone in questo caso di un cluster di server eseguiti all'interno di container e, in un contesto più ampio, di un cluster Kubernetes.

3.2 Tecnologie utilizzate

3.2.1 Python

Python è un linguaggio di programmazione *general-purpose* orientato agli oggetti e ad alto livello, molto diffuso in ambiti quali l'analisi dei dati, l'intelligenza artificiale, la finanza e la sicurezza informatica[9, pp. 7–8]. Pone grande enfasi sulla leggibilità e dispone di una sintassi particolarmente semplice che, insieme a una vastissima gamma di librerie, permette una grande riduzione in termini di linee di codice[14].

3.2.2 PostgreSQL

PostgreSQL è un Database Management System (DBMS) di tipo relazionale e ad oggetti, molto potente ed open source. Si tratta di una valida alternativa ad altri progetti sia liberi che con licenza, ed è divenuto celebre nel tempo per alcuni suoi punti di forza, primo fra tutti la programmabilità, che permette un incremento nel livello di astrazione, di affidabilità e prestazioni[15, 16].

3.2.3 Docker

Docker è una piattaforma aperta per lo sviluppo, la consegna e l'esecuzione di applicazioni all'interno di container. All'interno di Docker sono definiti essenzialmente due oggetti principali:

- *Immagini*: Un'immagine è un template contenente le istruzioni per generare un nuovo container. La generazione di nuove immagini è estremamente semplice ed avviene sulla base di istruzioni definite in un *Dockerfile*, un file contenente le istruzioni per la generazione di un ambiente preconfigurato e pronto per l'esecuzione. Ogni istruzione all'interno del Dockerfile corrisponde a un livello nella generazione dell'immagine, facilitando la condivisione e la distribuzione della stessa; quando un Dockerfile viene modificato e l'immagine è ricompilata, solo i livelli che hanno subito modifiche vengono ricompilati. Si tratta quindi di strutture molto leggere, piccole e veloci se comparate ad altri modelli di virtualizzazione[2].
- *Container*: L'idea alla base dei container è già stata spiegata nella Sezione 2.4, nel contesto di Docker i container assumono il ruolo di istanze eseguibili delle immagini[2].

Docker risulta ottimo per la gestione di container su un singolo host, offrendo strumenti per la limitazione e il monitoraggio delle risorse o il bilanciamento del carico (attraverso *Docker Swarm* o *Overlay Network Driver*), con caratteristiche che risultano tuttavia limitate rispetto ad altri orchestratori più avanzati[13].

3.2.4 Kubernetes

Kubernetes è una piattaforma open source per l'automazione della distribuzione, della scalabilità e della gestione di applicazioni eseguite su container, azioni che nell'insieme prendono il nome di *orchestrazione*. La necessità di orchestratori di container nasce in contesti di produzione, laddove c'è la necessità di gestire un gran numero di container ed assicurare che i servizi offerti siano sempre attivi[2].

Alcune delle caratteristiche principali di Kubernetes sono:

- *Gestione delle risorse*: Permette di porre limiti sulle risorse utilizzabili da un container (RAM, CPU, disco, etc.).
- *Bilanciamento del carico*: È in grado di gestire più repliche di uno stesso container e gestire il traffico in ingresso distribuendolo uniformemente sulle istanze attive. Fornisce inoltre la funzione di *auto-scaling* che permette di variare dinamicamente il numero di istanze di un container sulla base del traffico di rete.
- *Self-healing* Termina automaticamente i container che si sono bloccati e li sostituisce con nuove istanze degli stessi.
- *Configurazione dichiarativa*: È possibile lavorare in modalità DSC (*Desired State Configuration*), in cui un cluster può essere configurato utilizzando dei file dichiarativi che descrivono lo stato in cui si vuole portare il sistema. La fase di transizione dallo stato attuale del cluster allo stato desiderato è del tutto trasparente all'utente e non è in alcun modo modificabile, sarà Kubernetes a prendere tutte le decisioni in merito evolvendo il sistema.

Gli elementi chiave di Kubernetes sono *Nodi*, *Cluster*, *Pod*, *Deployment*, *Service*. Di seguito una breve presentazione per ognuno di essi:

Cluster Un cluster è l'unità logica più ampia definita in Kubernetes e rappresenta l'ambiente all'interno del quale sono orchestrati i container.

Nodi I nodi possono essere macchine fisiche o virtuali e rappresentano i punti tra i quali è suddiviso il carico di lavoro. Ne esistono due tipi: i *control plane node* e i *worker node*; i primi gestiscono l'interazione con il cluster e si occupano di distribuire uniformemente il carico di lavoro tra i *worker node*; i secondi sono nodi operativi, sui quali vengono istanziati i *pod*[3].

Pod Un Pod è la più piccola unità definibile con Kubernetes e corrisponde a un gruppo di uno o più container che condividono spazio di archiviazione e risorse di rete. Ogni pod genera di fatto un host logico[4].

Deployment e Replicaset Un *Deployment* è un oggetto Kubernetes che permette di dichiarare come le applicazioni al suo interno debbano essere gestite, distribuite e quante risorse potrà impegnare. All'interno del deployment viene solitamente definito un oggetto *ReplicaSet*, il quale racchiude un insieme di pod che condividono una configurazione comune. La risorsa *ReplicaSet* si occupa di mantenere il numero di repliche desiderato per ciascun pod, istanziandone di nuovi o distruggendo quelli bloccati. Un Deployment è un concetto di più alto livello che permette di gestire i ReplicaSet e fornire aggiornamenti sulla configurazione dei pod in maniera dichiarativa[5]; è ad esempio possibile specificare le immagini da utilizzare per la generazione di container, i numeri di porta da esporre, etc.

Service Un oggetto *Service* definisce uno strumento utile ad esporre un gruppo di pod sulla rete. La necessità di definire un tale oggetto nasce dalla natura effimera dei pod, che come precedentemente spiegato possono essere distrutti e rigenerati; da ciò ne consegue che non è possibile conoscere quali unità sono attive e funzionanti in un dato momento. La configurazione di un service contiene il nome del service stesso, un selettore per identificare i pod raggiungibili e i numeri di porta accessibili dall'esterno, generando di fatto un'astrazione dagli indirizzi IP interni di un insieme di pod[6].

3.2.5 API REST

Per API REST (o RESTful) si intende una *Application Programming Interface* progettata secondo i principi dello stile architetturale REST (*REpresentational State Transfer*). I principi guida di REST si articolano in sei punti e sono definiti con l'obiettivo di promuovere semplicità e scalabilità:

1. *Interfaccia uniforme*: Ogni risorsa trasferita dal server deve rispettare delle condizioni di generalità: identificazione univoca della risorsa, manipolazione della risorsa tramite rappresentazioni (le risorse devono essere rappresentate in un formato standard come JSON o XML), autodescrizione delle risorse.
2. *Architettura client-server*: Client e server possono essere sviluppati separatamente, in modo indipendente uno dall'altro.
3. *Stateless*: Ogni richiesta dal client al server deve contenere tutte le informazioni necessarie per comprendere e completare la richiesta, il server non deve far uso di informazioni memorizzate precedentemente.

4. *Cache*: Ogni risorsa deve specificare se può essere memorizzata e riutilizzata in seguito come risposta a una richiesta equivalente.
5. *Sistema a più livelli*: È possibile definire una struttura gerarchica tra livelli, così che all'arrivo di una richiesta il server principale possa inoltrare quest'ultima ad altri server di livello inferiore. Solo il livello più esterno deve però essere visibile all'utente.
6. *Codice on demand*: Oltre a risorse statiche, all'occorrenza può essere inviato anche del codice sorgente.

Per finire, REST non impone l'utilizzo di alcun protocollo di comunicazione, sebbene HTTP sia la scelta più comune[7].

3.2.6 Token JWT

JWT (*JSON Web Token*) è uno standard aperto che definisce una modalità compatta ed autocontenuta adatta a trasmettere informazioni tra le parti coinvolte in una comunicazione come oggetti JSON. I dati inviati sono firmati digitalmente grazie ad algoritmi a chiave segreta (HMAC) o a chiave pubblica-privata (RSA) che permettono di verificarne facilmente l'autenticità.

I token JWT si compongono di tre parti separate tra loro da un punto ("."): *header*, *payload* e *firma*. L'header contiene solitamente l'algoritmo usato per generare la firma e il tipo di token; il payload contiene i *claims*, ossia delle proprietà legate alle entità coinvolte; il campo firma contiene l'*impronta hash* generata dai dati delle due parti precedenti utilizzando l'algoritmo scelto.

I vantaggi nell'utilizzo di JWT, se confrontato con altri sistemi per la generazione di web token, risiedono nella compattezza dei pacchetti, nella flessibilità nell'uso di diversi algoritmi di firma e nella diffusione e semplicità di manipolazione di oggetti JSON[8].

4

Implementazione

Le due applicazioni sviluppate mirano a soddisfare le stesse richieste e presentano perciò una similarità molto stretta a livello di tecnologie utilizzate. Nel corso di questo capitolo verrà presentata dapprima l'applicazione monolitica, analizzandone i servizi offerti, la struttura interna e le scelte progettuali compiute; in seguito si passerà all'analisi del sistema a microservizi, concentrandosi sul processo di decomposizione del problema generale.

L'applicazione si compone complessivamente 5 servizi

1. *Autenticazione*: Autentica gli utenti e verifica la validità dei token presentati.
2. *Gestione clienti*: Permette la registrazione di nuovi clienti.
3. *Gestione ordini*: Permette la generazione di nuovi ordini.
4. *Gestione pagamenti*: Gestisce il pagamento degli ordini.
5. *Spedizioni*: Permette di generare nuove spedizioni e verificare lo stato di una spedizione esistente.

4.1 Autenticazione e autorizzazione

Gli utenti devono poter compiere solo le operazioni per le quali sono autorizzati sulla base del proprio tipo di utente, un cliente non deve per esempio poter rifornire il magazzino. Per questo motivo ad ogni richiesta inviata tramite API viene aggiunto un token JWT

con quattro *claim* privati: **badge_n**, il numero di tessera del cliente, **created**, data e ora di creazione, **expires**, data e ora di scadenza, **role**, tipologia di utente.

Il token è rilasciato durante la fase di login ed ha una validità di trenta minuti, superata la quale occorre autenticarsi nuovamente. Il processo di login segue i seguenti passi:

1. L'applicazione riceve una richiesta POST sul percorso `/login` in cui sono forniti email e hash della password (generato secondo l'algoritmo SHA-3) dell'utente che richiede di autenticarsi.
2. Viene interrogato il database e vengono recuperate le informazioni associate all'utente sulla base dell'email fornita.
3. Se lo hash presente nel database e quello fornito coincidono, viene generato il token inserendo tutti i claim sopra elencati.
4. Il token viene firmato e restituito al client.

Il token è firmato utilizzando l'algoritmo a chiave segreta HS256, procedura fondamentale al fine di garantire l'autenticità del token stesso quando questo sarà ripresentato.

4.2 Database

Il database si compone di sei tabelle (vedi Figura 4.1):

- *Articles*: modella il magazzino, contiene i prodotti e le relative quantità.
 - *barcode*: codice a barre univoco dell'articolo;
 - *name*: nome dell'articolo;
 - *quantity*: quantità presente in magazzino;
 - *unit_price*: costo unitario del prodotto.

Per la colonna *quantity* si è definito il vincolo `positive_quantity_remaining_check`, il quale impedisce che la quantità residua di un prodotto assuma valori negativi.

- *Customers*: elenco dei clienti registrati.
 - *customerid*: id univoco dell'utente;
 - *surname*: cognome;
 - *badge_n*: numero di tessera;
 - *sex*: sesso;

- *dob*: data di nascita;
 - *email*: email;
 - *password_hash*: codice hash della password;
 - *customerrole*: tipologia di utente (vedi tabella successiva).
- *OrderItems*: contiene gli articoli acquistati in ogni ordine.
 - *articleid*: codice a barre dell'articolo acquistato;
 - *orderid*: id dell'ordine in cui è stato acquistato l'articolo;
 - *quantity*: quantità acquistata.
- *Orders*: contiene gli ordini effettuati.
 - *orderid*: id univoco dell'ordine;
 - *badge_n*: numero di tessera del cliente che ha effettuato l'ordine;
 - *orderdate*: data ed ora in cui è stato generato l'ordine;
 - *total_price*: prezzo complessivo dell'ordine;
 - *orderstate*: stato dell'ordine (vedi tabella successiva).
- *orderstate*: contiene gli stati definiti per un ordine. Essi sono rappresentati in una tabella a parte al fine di garantire coerenza nel database e facilità di comprensione e manutenzione.
 - *id*: numero intero associato allo stato;
 - *statename*: nome dello stato;
- *Roles*: contiene i ruoli definiti per gli utenti. Valgono le stesse considerazioni fatte per la tabella precedente.
 - *rolename*: nome del ruolo

4.3 Esecuzione delle richieste

Le richieste ricevute sono eseguite nel seguente modo:

1. Vengono verificati la validità del token fornito e i permessi dell'utente ad effettuare la richiesta, se uno di questi due controlli fallisce viene inviata direttamente una risposta di errore.

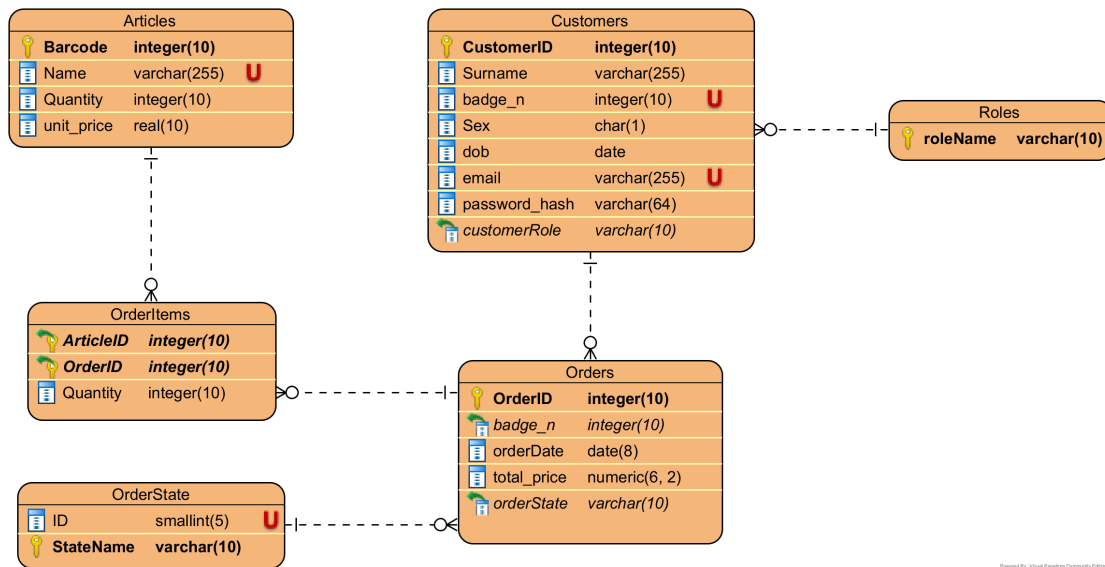


Figura 4.1: Schema del database

- La gestione della richiesta viene demandata alla classe specifica, che interroga il database se necessario e predispone un oggetto JSON di risposta. Ogni classe dispone dei permessi in scrittura sul database; si è compiuta questa scelta al fine di evitare i colli di bottiglia tipici di una gestione centralizzata.
- Viene generata la risposta, restituendo la risorsa richiesta in formato JSON insieme a un codice di stato (*status code*). Il codice di stato è utile per identificare l'esito positivo o negativo della richiesta, inserendo eventualmente l'errore all'interno di una macro categoria.

4.4 Evoluzione dello stato di un ordine

Sono definiti sei stati in cui può trovarsi un ordine, si riporta il diagramma a stati finiti in Figura 4.2.

- Created*: L'ordine è stato ricevuto.
- Confirmed*: L'ordine può essere soddisfatto, i prodotti acquistati sono stati registrati.
- Cancelled*: Non è possibile soddisfare l'ordine, almeno un prodotto è presente a magazzino in quantità insufficiente.
- Paid*: È stato ricevuto il pagamento.

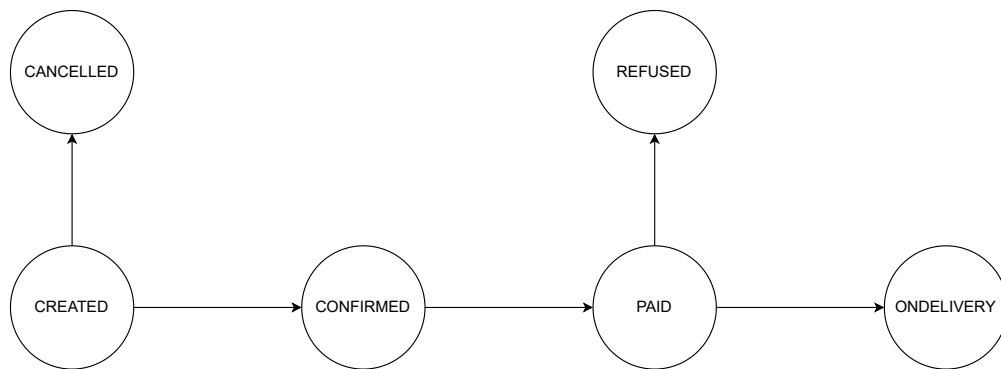


Figura 4.2: Diagramma a stati finiti degli stati definiti per un ordine

- *Refused*: Il pagamento è stato rifiutato.
- *OnDelivery*: L'ordine è stato evaso e gli articoli spediti.

Il processo di creazione di un nuovo ordine ha inizio con l'arrivo di una richiesta POST all'API sul percorso `/newOrder` e può essere effettuato solamente da utenti di tipo *customer*. Il corpo della richiesta deve essere in formato JSON e contenere un array di prodotti selezionati e la relativa quantità; la struttura deve essere quella riportata nel Codice 1.

Non è necessario specificare l'identificativo dell'utente, in quanto questa informazione, insieme al tipo di utente, è già presente all'interno del token ed è recuperata automaticamente. I passaggi successivi sono:

1. Viene registrato il nuovo ordine nella tabella *Orders* del database, il quale si occupa di assegnare un ID univoco. Lo stato dell'ordine è impostato su *created* finché non ha l'ID assegnato e viene in seguito modificato in *confirmed*.
2. Vengono prelevati i prodotti ordinati dal magazzino, modificando le quantità residue nella tabella *Articles*. Se si tenta di rimuovere un numero di prodotti superiore a quelli presenti in magazzino viene generata un'eccezione per la violazione del *vincolo check* spiegato alla Sezione 4.2; il database esegue il rollback e lo stato dell'ordine è impostato su *Cancelled*.
3. Vengono registrati tutti i prodotti ordinati con la quantità associata nella tabella *OrderItems* del database.
4. Viene calcolato il prezzo finale dell'ordine e memorizzato nella tabella *Orders*.

Se tutto va a buon fine l'ordine è modificato in *Confirmed* e all'utente viene comunicata la somma da pagare.

```
[
  {
    "barcode": 3,
    "name": "whole milk",
    "quantity": 2
  },
  {
    "barcode": 74,
    "name": "oil",
    "quantity": 1
  },
  {
    // ...
  }
]
```

Listing 1: Struttura del corpo della richiesta per la creazione di un nuovo ordine

4.5 Pagamento di un ordine

La funzione di pagamento degli ordini è molto semplificata e puramente esemplificativa, la connessione con un vero servizio di pagamento elettronico avrebbe aumentato di molto la complessità del progetto, uscendo dagli obiettivi dello stesso. Si è quindi deciso di simulare le operazioni di pagamento *addormentando* il thread per 100 ms.

Per richiamare la funzione di pagamento è sufficiente inviare una richiesta sul percorso `/payOrder`, includendo il token di autenticazione all'interno del *header* della chiamata e un file JSON contenente l'ID dell'ordine che si desidera completare. La forma deve essere la seguente `{"orderId": 16}`.

4.6 Implementazione dei microservizi

L'applicazione a microservizi nasce dal codice dell'applicazione monolitica riorganizzandolo in unità indipendenti sulla base dei diversi servizi offerti. Al fine di evitare colli di bottiglia dovuti alla presenza di un singolo punto d'ingresso per le richieste, ogni microservizio espone delle proprie API su un percorso e una porta specifici.

Sono stati sviluppati sei microservizi, lo schema generale del sistema è mostrato in Figura 4.3. Di seguito l'elenco dei microservizi:

- *Authentication*: Gestisce l'autenticazione dell'utente, rilasciando e verificando la validità dei token. Espone API con metodo POST sul percorso `/login` alla

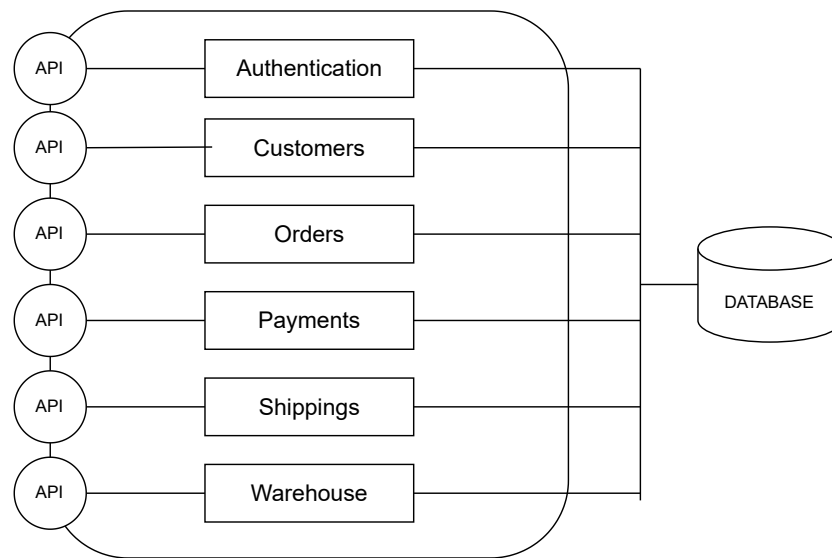


Figura 4.3: Struttura del sistema a microservizi

porta 5000.

- *Customers*: Permette l'iscrizione di nuovi utenti, tramite una API con metodo POST alla porta 5005 sul percorso `/registerCustomer`.
- *Orders*: Gestisce il processo di creazione di un nuovo ordine. Espone API con metodo POST alla porta 5002 sul percorso `/newOrder`.
- *Payments*: Gestisce la procedura di pagamento. Espone API con metodo POST alla porta 5003 sul percorso `/payOrder`.
- *Shippings*: Genera nuove spedizioni e permette di controllare lo stato di una spedizione. Espone API con metodo POST alla porta 5004 sul percorso `/trackOrder`.
- *Warehouse*: Gestisce il magazzino. Ha una propria API attiva sulla porta 5001 al percorso `productsList`, con metodo GET per richiedere la lista dei prodotti presenti in magazzino, POST per caricare nuovi prodotti.

Sempre al fine di evitare colli di bottiglia si è preferito lasciare il DBMS in esecuzione locale, senza creare un microservizio dedicato che avrebbe rappresentato un ulteriore grado di complessità.

L'applicazione a microservizi è stata eseguita su container utilizzando Kubernetes come orchestratore nella sua versione completa, solitamente riferita anche come K8s (8 è il numero di lettere tra *K* e *s*). L'avvio di un cluster Kubernetes non è un processo semplice e implica diversi passaggi.

```
apiVersion: v1
kind: Service
metadata:
  name: authentication-service
spec:
  selector:
    app: authentication-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
```

Listing 2: Dichiarazione del service per il deployment Authentication

- Occorre scegliere l'infrastruttura su cui eseguire il cluster, per questo progetto è stato scelto di avviare il tutto in locale.
- Bisogna installare un *sistema runtime* per container, ossia il sistema che si occuperà dell'esecuzione di ciascun container; Docker in questo caso.
- Occorre installare *kubectl*, *kubeadm*, *kubelet* e *kubernetes-cni*, componenti chiave di Kubernetes che permettono l'interazione, la creazione e il mantenimento del cluster kubernetes.
- Infine, si può procedere con l'istanziamento del cluster vero e proprio, che nel caso in esame è avvenuto fornendo tramite shell al comando *kubectl* un file con la configurazione desiderata in formato YAML (come spiegato alla Sezione 3.2.4).

La configurazione ha previsto la generazione di sei *Deployment*, ciascuno dedicato all'esecuzione dei pod relativi a un microservizio; il *ReplicaSet* di tutti i pod è stato impostato a 1 poiché il sistema si è rivelato in grado di soddisfare tutte le richieste in tempi brevi durante la fase di test. Parallelamente ad ogni deployment, all'interno del file di configurazione si è anche specificata la generazione di sei *Service*, tutti di tipo *Load Balancer*, la categoria che permette l'esposizione di alcune porte all'esterno del cluster e la distribuzione del numero di richieste su tutte le istanze di un'applicazione. In Listing 2 è riportato un esempio di dichiarazione di un service.

In Listing 4.1 è riportato lo stato di Kubernetes con l'applicazione a microservizi in esecuzione.

4.7 Altre funzionalità

Le rimanenti funzionalità dell'applicazione riguardano la registrazione di nuovi clienti e il tracciamento dei pacchi.

La registrazione di nuovi clienti avviene attraverso un'API sul percorso `/registerCustomer` inviando un file JSON con i campi *cognome*, *Sesso*, *data di nascita*, *email*, *password*. Se tutte le informazioni rispettano i vincoli relativi all'attributo in esame l'utente viene registrato ed è subito possibile effettuare il login.

La funzionalità di tracciamento dei pacchi è invocabile sul percorso `/trackOrder` dell'API specificando come parametro all'interno dell'URL l'ID dell'ordine di cui si vuole conoscere lo stato. La chiamata all'API avverrà con un URL di questa forma: `localhost:5000/trackOrder?id=16`.

```

PS C:\Users\...> kubectl get deployment
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
authentication-depl                1/1      1              1             7s
customers-depl                     1/1      1              1             7s
orders-depl                        1/1      1              1             7s
payments-depl                      1/1      1              1             7s
shippings-depl                    1/1      1              1             7s
warehouse-depl                    1/1      1              1             7s

PS C:\Users\...> kubectl get service
NAME                                TYPE            CLUSTER-IP      EXTERNAL-IP
PORT(S)          AGE
authentication-service  LoadBalancer    10.96.111.66     localhost
5000:31843/TCP      34s
customers-service      LoadBalancer    10.102.99.111    localhost
5005:30826/TCP       34s
kubernetes            ClusterIP        10.96.0.1        <none>
443/TCP              8h
orders-service         LoadBalancer    10.100.51.68     localhost
5002:30142/TCP        34s
payments-service       LoadBalancer    10.108.26.17     localhost
5003:32284/TCP        34s
shippings-service      LoadBalancer    10.111.29.183    localhost
5004:31344/TCP        34s
warehouse-service      LoadBalancer    10.108.154.128   localhost
5001:32499/TCP        34s

PS C:\Users\...> kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
authentication-depl-7d87856d8d-p98bz 1/1      Running   0            63s
customers-depl-cc9c9bdfb-dpmw5       1/1      Running   0            63s
orders-depl-5896b9c76f-5fwkk         1/1      Running   0            63s
payments-depl-55746f8454-7zdzk       1/1      Running   0            63s
shippings-depl-59f766cf5-1klxs       1/1      Running   0            63s
warehouse-depl-86b6988d74-pxwlg      1/1      Running   0            63s

PS C:\Users\...> kubectl get replicaset
NAME                                DESIRED    CURRENT    READY    AGE
authentication-depl-7d87856d8d      1          1          1        88s
customers-depl-cc9c9bdfb            1          1          1        88s
orders-depl-5896b9c76f              1          1          1        88s
payments-depl-55746f8454            1          1          1        88s
shippings-depl-59f766cf5            1          1          1        88s
warehouse-depl-86b6988d74           1          1          1        88s

```

Listing 4.1: Stato di Kubernetes durante l'esecuzione dell'applicazione a microservizi. In ordine dall'alto al basso sono mostrati deployment, service, pod, replicaset

5

Risultati ottenuti

Il sistema è stato avviato in locale, su un computer che funge sia da nodo *master* che da nodo *worker*. Al fine di analizzare le performance dei due sistemi (monolitico e a microservizi) è stato realizzato uno script che effettua mille ordini in successione. Le operazioni eseguite nel dettaglio sono le seguenti:

1. Viene selezionato in modo casuale un utente con il quale si effettua il login.
2. Viene richiesta la lista articoli disponibili. Dalla lista vengono selezionati alcuni prodotti in modo casuale (da 1 a 15 prodotti diversi), ciascuno in quantità casuale tra 1 e 10; si genera così un carrello virtuale.
3. Viene inviato l'ordine con i prodotti scelti.
4. L'ordine viene pagato.
5. Se non si sono generati errori la procedura è completata e si passa all'esecuzione della prossima iterazione.

Ad ogni iterazione lo script memorizza l'intervallo di tempo in sei differenti punti:

1. Inizio della nuova iterazione.
2. Completato il login dell'utente.
3. Ottenuta la lista di prodotti dal server.

4. Carrello riempito con i prodotti selezionati secondo la modalità precedentemente descritta.
5. Il nuovo ordine è stato generato.
6. Fine.

Si possono quindi calcolare quattro intervalli di tempo, indicati in ordine con lettere da A a D. In Tabella 5.1 e 5.2 sono riportati gli indici statistici più rilevanti ottenuti dai dati inerenti i tempi di esecuzione di ogni fase, in Figura 5.1, 5.2, 5.3 e 5.4 i dati sono invece rappresentati sotto forma di boxplot ed istogramma.

Gli indici di maggior interesse sono sicuramente media e mediana, i quali evidenziano un generale leggero vantaggio per l'applicazione monolitica; la versione a microservizi risenti infatti dell'*overhead* introdotto dall'orchestratore dei container.

In un'applicazione semplice come questa il sistema monolitico presenta un vantaggio poiché si presenta complessivamente più snello, il crescere del traffico da soddisfare farebbe tuttavia emergere i vantaggi di un sistema a microservizi e dell'esecuzione con un orchestratore quale Kubernetes, permettendo la distribuzione delle richieste su più nodi.

Per concludere va anche tenuto in considerazione che il test è stato eseguito su un personal computer di potenza limitata, sicuramente ben lontano dalla capacità di calcolo offerta da un server reale.

Anche un secondo script è stato realizzato al fine di emulare un cliente di tipo *supplier* (fornitore); questo effettua periodicamente (ogni 15 secondi) il login con le proprie credenziali e richiede la lista dei prodotti a magazzino (tramite una richiesta POST sul percorso `/productsList`), aggiungendo una quantità casuale dei prodotti presenti con meno di 50 pezzi. Questa operazione è molto utile in quanto permette di rifornire costantemente il magazzino virtuale assicurando che il gran numero di ordini effettuati dagli utenti possa essere soddisfatto. Nella logica applicativa infatti la mancanza di anche solo un prodotto porta alla cancellazione dell'intero ordine effettuato dal cliente.

Fase	A[ms]	B[ms]	C[ms]	D[ms]
Minimo	29.86	29.76	33.61	128.92
1° quartile	37.52	36.75	43.02	136.58
Mediana	40.09	39.39	46.33	138.96
3° quartile	43.3	42.05	49.75	142.35
Massimo	171.75	163.46	170.81	367.56
Media	42.86	41.35	48.88	57.24

Tabella 5.1: Valori statistici ottenuti dall'esecuzione dell'applicazione monolitica

Fase	A[ms]	B[ms]	C[ms]	D[ms]
Minimo	34.89	33.6	44.92	138.89
1° quartile	39.62	36.17	51.56	146.91
Mediana	41.43	37.44	54.83	148.99
3° quartile	43.64	39.14	58.11	151.26
Massimo	201.69	200.33	206.92	278.27
Media	44.05	40.64	58.31	152.51

Tabella 5.2: Valori statistici ottenuti dall'esecuzione dell'applicazione a microservizi

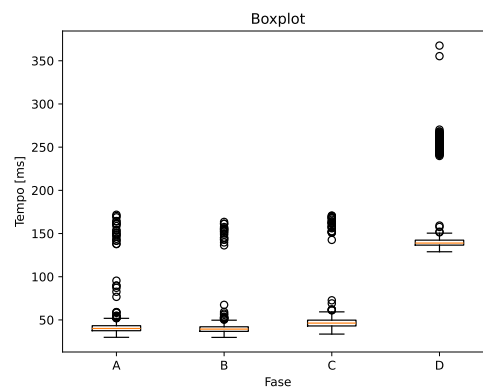


Figura 5.1: Boxplot relativo ai tempi di esecuzione dell'applicazione monolitica

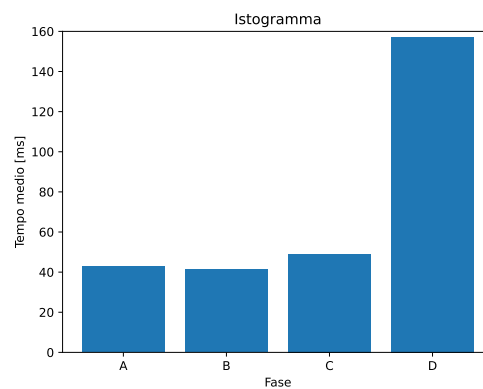


Figura 5.2: Istogramma relativo ai tempi di esecuzione dell'applicazione monolitica

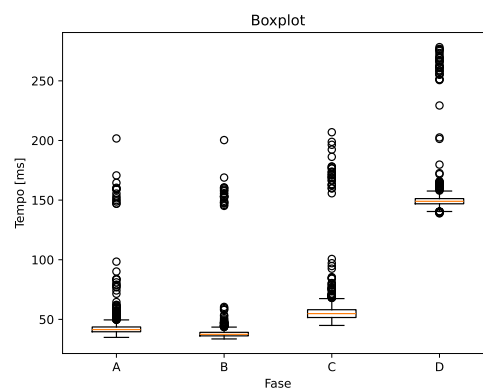


Figura 5.3: Boxplot relativo ai tempi di esecuzione dell'applicazione a microservizi

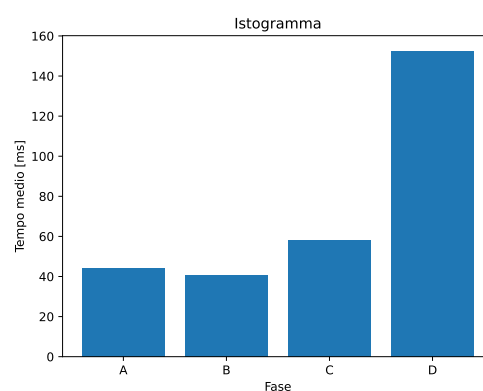


Figura 5.4: Istogramma relativo ai tempi di esecuzione dell'applicazione a microservizi

6

Conclusioni e sviluppi futuri

In questo lavoro di tesi si è compiuto un confronto tra due applicazioni realizzate seguendo approcci alla progettazione differenti, da un lato la modalità tradizionale di sviluppo, dall'altra una corrente emergente in crescita di anno in anno.

L'utilizzo di Kubernetes rende più efficiente lo sviluppo e facilita la gestione del sistema, da sottolineare sono in particolari le caratteristiche di *self-healing* e scalabilità offerte.

Gli sviluppi futuri potrebbero essere molteplici e spaziare dall'ambito della sicurezza alla migrazione verso nuove tecnologie. L'aspetto sicurezza è indubbiamente di primaria importanza, le applicazioni sviluppate espongono infatti alla possibilità che un utente malevolo riesca ad impossessarsi di un token destinato a un altro utente, utilizzandolo per nascondere la propria identità. In secondo luogo la migrazione da PostgreSQL ad un database NoSQL porterebbe a un incremento della velocità di esecuzione delle query e permetterebbe la scalabilità orizzontale del sistema, aspetto di primaria importanza per un sistema distribuito. Valide alternative potrebbero essere MongoDB o Cassandra. Infine si potrebbe riscrivere l'intera applicazione in un linguaggio compilato: sebbene Python sia un linguaggio moderno, flessibile e sintetico, esso risulta essere uno dei linguaggi più lenti in esecuzione a causa della presenza dell'interprete; inoltre si presta bene solo per la scrittura di script o piccole applicazioni, altri linguaggi offrono un supporto molto maggiore alla programmazione in grande (C++, Java, Rust, etc). L'utilizzo di un linguaggio compilato porterebbe in questo senso ad un buon incremento delle prestazioni.

Elenco delle figure

2.1	Schema di funzionamento di un'applicazione monolitica[10]	4
2.2	Schema di funzionamento di un'applicazione a microservizi[10]	4
2.3	Confronto tra un sistema basato su macchine virtuali e container[11]	6
4.1	Schema del database	15
4.2	Diagramma a stati finiti degli stati definiti per un ordine	16
4.3	Struttura del sistema a microservizi	18
5.1	Boxplot relativo ai tempi di esecuzione dell'applicazione monolitica	24
5.2	Istogramma relativo ai tempi di esecuzione dell'applicazione monolitica	25
5.3	Boxplot relativo ai tempi di esecuzione dell'applicazione a microservizi	25
5.4	Istogramma relativo ai tempi di esecuzione dell'applicazione a microservizi	25

Elenco delle tabelle

5.1	Valori statistici ottenuti dall'esecuzione dell'applicazione monolitica	24
5.2	Valori statistici ottenuti dall'esecuzione dell'applicazione a microservizi . .	24

Bibliografia

- [1] *Overview on Kubernetes*. URL: <https://kubernetes.io/docs/concepts/overview/>.
- [2] *Docker overview*. URL: <https://docs.docker.com/get-started/overview/>.
- [3] *Kubernetes nodes*. URL: <https://kubernetes.io/docs/concepts/architecture/nodes/>.
- [4] *Pods – Pods are the smallest deployable units of computing that you can create and manage in Kubernetes*. URL: <https://kubernetes.io/docs/concepts/workloads/pods/>.
- [5] *ReplicaSet – A ReplicaSet’s purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of a specified number of identical Pods*. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>.
- [6] *Service – In Kubernetes, a Service is a method for exposing a network application that is running as one or more Pods in your cluster*. URL: <https://kubernetes.io/docs/concepts/services-networking/service/#endpoints>.
- [7] *What is REST*. URL: <https://restfulapi.net/>.
- [8] *Introduction to JSON Web Tokens*. URL: <https://jwt.io/introduction>.
- [9] Mark Lutz. *Learning Python*. O’Reilly Media, 2009. ISBN: 978-0-596-15806-4.
- [10] *Microservices vs. monolithic architecture – When monoliths grow too big it may be time to transition to microservices*. URL: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>.
- [11] *Containers vs. virtual machines – Learn the differences between containers and virtual machines, popular providers for each, and how they can be used together*. URL: <https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms>.
- [12] *What is Podman?* URL: <https://www.redhat.com/en/topics/containers/what-is-podman>.

-
- [13] *Kubernetes vs Docker: A comprehensive comparison*. URL: <https://www.civo.com/blog/kubernetes-vs-docker-a-comprehensive-comparison#:~:text=Improved%20container%20orchestration%3A%20Kubernetes%20provides,demand%2C%20ensuring%20efficient%20resource%20utilization..>
- [14] *Python (programming language)*. URL: [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)).
- [15] *PostgreSQL – About*. URL: <https://www.postgresql.org/about/>.
- [16] *PostgreSQL*. URL: <https://it.wikipedia.org/wiki/PostgreSQL>.